ARIA: Autonomous Reinforcement Irrigation Agent

Tiya (UEN: 135)
School of Computer Science
B.Tech Data Science
April 30, 2026

Vaidhav (UEN: 136)
School of Computer Science
B.Tech Data Science
April 30, 2026

I. Introduction

A. Background

Irrigation accounts for roughly 70% of global freshwater consumption. Despite this, most irrigation systems in use today operate on fixed timers. They water crops on a schedule regardless of whether it is raining, whether the soil is already saturated, or whether a drought is stressing the plant. This kind of blindness leads to both overwatering and underdog, causing water waste, hardware damage from overworked pumps, and reduced crop yields.

B. Motivation

Climate change is making weather patterns increasingly unpredictable. Droughts arrive earlier, monsoons arrive harder, and the window for optimal crop growth is narrowing. At the same time, low cost IoT sensors and affordable computing hardware have made real-time AI-controlled agriculture genuinely feasible. The tools exist to do better. The question is whether an AI agent can learn, from experience, what the right irrigation decision is at any point in a crop's lifecycle.

C. Why Reinforcement Learning

Reinforcement Learning is a natural fit for this problem. A crop's lifecycle is a long sequential decision making task where the consequences of actions (watering or not watering) play out over days and weeks rather than seconds. The agent cannot be told the correct answer explicitly because the correct amount of water depends on current soil moisture, temperature, humidity, how much rain is forecast, how much water is left in the tank, and how far along the plant is in its growth cycle. RL allows the agent to discover this policy on its own through repeated interaction with the environment, receiving reward signals that guide it toward better behavior over time.

Traditional rule based systems cannot generalize across all these variables simultaneously. Supervised learning would require labeled examples of "correct" irrigation decisions, which are difficult to define and collect. RL requires only a well-designed environment and a reward function, and it learns the rest.

Final Project Report for Course: Reinforcement Learning (DATA405), Semester VI, B.Tech Data Science.

II. Problem Formulation

A. Environment Description

The environment, FarmEnv is a custom simulation built to mimic the physical conditions of a tomato farm over a 90 day growing season. It is structured as a Markov Decision Process where each timestep represents one day. The environment simulates three seasonal phases: Spring (days 0 to 30) with mild temperatures around 25°C and 50% humidity, a Summer Drought (days 30 to 60) with temperatures rising to 35°C and humidity dropping to 20%, and a Monsoon (days 60 to 90) where temperatures cool to 22°C and humidity rises to 80% with heavy rainfall. Each phase is randomized using Gaussian noise so the agent cannot memorize a fixed pattern.

B. State Space

The agent observes an 8 dimensional state vector at each timestep. All values are normalized to the range 0 to 1 to prevent gradient issues during neural network training.

TABLE I
State Space Description

Dimension	Description
Soil Moisture	Current water content of the soil (optimum at 0.55)
Water Tank Level	Remaining volume in the physical reservoir
Growth Score	Cumulative biological mass of the plant so far
Normalized Day	Current day divided by 90
Temperature	Ambient temperature after normalization
Humidity	Atmospheric humidity after normalization
Rain Now	Current precipitation level
Rain Forecast	Anticipated rainfall over the next 24 hours

C. Action Space

The action space is discrete with two options. Action 0 means do nothing. Action 1 means irrigate which adds 20% moisture to the soil and deducts 5% from the water tank. This binary design maps directly to the physical relay switch on the ESP32 microcontroller in the real deployment.

D. Reward Function

The reward function was designed through several iterations to prevent the agent from finding shortcuts that maximize its score while failing the actual goal. The final structure uses six signals:

TABLE II
REWARD FUNCTION SIGNALS

Signal	Value	Trigger
Growth below expected	-5.0	Actual growth ¡ expected
Growth above expected	$+\delta$	Actual growth ¿ expected
Pump activation cost	-0.05	Every irrigation action
Empty tank pump attempt	-5.0	Tank below 5%
Dry soil (no irrigation)	-2.0	Soil below 30%, action = 0
Harvest bonus	+100.0	Day 89, growth $\geq$ 80%

The large harvest bonus at the end of the episode is what forces the agent to think long term. Without it the mathematically cheapest strategy is to simply let the plant die on Day 1 and avoid all future water penalties.

E. Episode Design

Each episode represents one full 90 day growing season. The environment resets at the start of each episode with a fresh plant, a full water tank and a newly randomized seasonal weather sequence. The agent must manage its water supply across all three seasons to keep the plant alive and growing above the expected biological curve so that it can collect the harvest reward on Day 89.

F. Assumptions and Constraints

The plant's biological growth is governed by a Weighted Generalised Mean equation using Gaussian curves. The only physical lever the agent controls is the water pump. Temperature and humidity are environmental variables the agent can observe but cannot influence. The water tank resets to 100% every 30 days simulating a human operator refilling the reservoir. During training live weather data is replaced by the randomized simulation so the agent learns general behavior rather than overfitting to one specific climate.

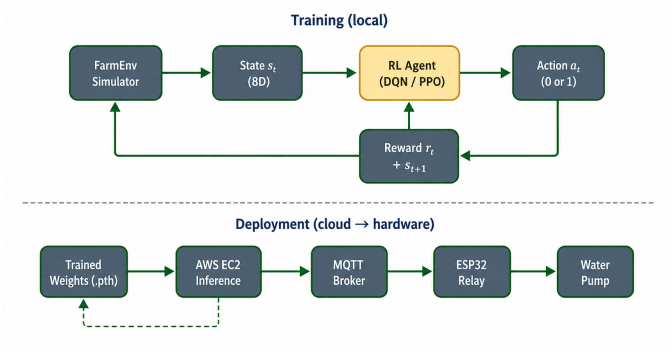

Fig. 1. System Workflow Diagram

III. METHODOLOGY

A. Algorithms Evaluated

Seven algorithms were trained and compared in the same environment:

TABLE III
ALGORITHM PERFORMANCE SUMMARY

Algorithm	Category	Outcome
Q-Learning	Tabular	Hit state-space ceiling
SARSA	Tabular	Hit state-space ceiling
REINFORCE (vanilla)	Policy Gradient	High variance, poor convergence
REINFORCE (baseline)	Policy Gradient	Starvation — never irrigated
Actor-Critic	Policy Gradient	Pump spammer — irrigated every step
DQN	Deep RL	Optimal policy
PPO	Deep RL	Near-optimal

B. DQN: How It Works

Deep QNetwork replaces the traditional Q table with a neural network. Instead of storing a value for every possible state action pair, the network takes the current 8D state as input and outputs estimated Q values for both actions. The key stability mechanisms are Experience Replay where the agent stores past transitions and trains on random batches rather than the most recent step and a Target Network which is a frozen copy of the main network used to compute learning targets so the agent is not chasing a constantly moving goal. DQN is off policy, meaning it can learn from experiences collected under a previous version of its policy.

C. PPO: How It Works

Proximal Policy Optimization is a policy gradient method that learns both a policy (what action to take) and a value function (how good the current state is). Its defining feature is a clipping mechanism that prevents the updated policy from changing too drastically in a single training step. This avoids catastrophic forgetting. PPO is on policy, so it can only train on data it recently collected making it slightly less sample efficient than DQN in this specific environment.

D. Why Tabular Methods Failed

QLearning and SARSA require discretizing the state space to build a Qtable. When continuous values like soil moisture at 0.553 or temperature at 0.412 are rounded into bins. The agent loses the precision needed to distinguish meaningfully different situations. Both agents plateaued around a reward of -100.

E. Exploration Strategy

DQN uses epsilon greedy exploration. The epsilon value decays from 1.0 down to a floor of 0.05 over training. PPO explores naturally because its actor network outputs a probability distribution over actions.

F. Hyperparameters

IV. IMPLEMENTATION DETAILS

A. Programming Language and Libraries

The system was built in Python 3.12 using **PyTorch** for neural networks and **NumPy** for data handling. **Paho MQTT** managed communication between the AWS EC2 inference server and the ESP32 hardware. **SQLite3** handled telemetry

TABLE IV
TRAINING HYPERPARAMETERS

Parameter	Value
Neural network architecture	8 inputs $\rightarrow$ 128 $\rightarrow$ 128 $\rightarrow$ 2 outputs
Activation functions	SiLU for DQN, ReLU for PPO
Learning rate	0.001 (Adam optimizer)
Discount factor (γ)	0.99
DQN batch size	64
DQN target sync	Every 10 episodes
DQN epsilon decay	0.995 per episode
PPO clipping parameter	0.2
PPO update epochs	4
Training episodes	500
Total simulated days	45,000

logging. **Ultralytics** and **OpenCV** powered the YOLO based tomato detection pipeline.

V. EXPERIMENTS AND RESULTS

A. Reward Curves

DQN converged to a final reward of -57.41. Its learning curve showed steep improvement in the first 150 episodes. PPO reached -73.58 with a noisier trajectory.

B. Agent Comparison Table

TABLE V
COMPARISON OF RL AGENTS

Algorithm	Final Reward	Avg Irrig.	Growth Dev.	Verdict
DQN	-57.41	45.0	0.14	Optimal
PPO	-73.58	47.2	0.21	Near-optimal
Q-Learning	-97.88	48.4	0.28	Tabular ceiling
SARSA	∼-105	∼49	∼0.30	Tabular ceiling
REINFORCE-van	∼-175	∼35	∼0.42	High variance
REINFORCE-bas	-338.29	0.0	0.62	Starvation
Actor-Critic	-300.96	90.0	—	Pump spammer

C. DQN's Learned Strategy

DQN learned a seasonal policy:

- **Spring:** Moderate pumping to build soil moisture.
- **Drought:** Increased frequency while tracking tank levels.
- **Monsoon:** Recognized rain signals and reduced pumping.

VI. DISCUSSION AND INSIGHTS

A. Reward Hacking

Failures exposed gaps in reward architecture. The **Actor Critic pump spammer** showed that agents exploit levers without hard resource constraints. The **REINFORCE starvation** showed that high daily penalties can discourage all action. **DQN stunting** occurred when absolute deviation penalties unintentionally punished success was fixed by splitting signals.

B. The State Space Hallucination Problem

Initial deployment failed due to a mismatch between the simulator's growth model (Gaussian) and the cloud script's approximation (Sigmoid). Even small numerical differences confused the network.

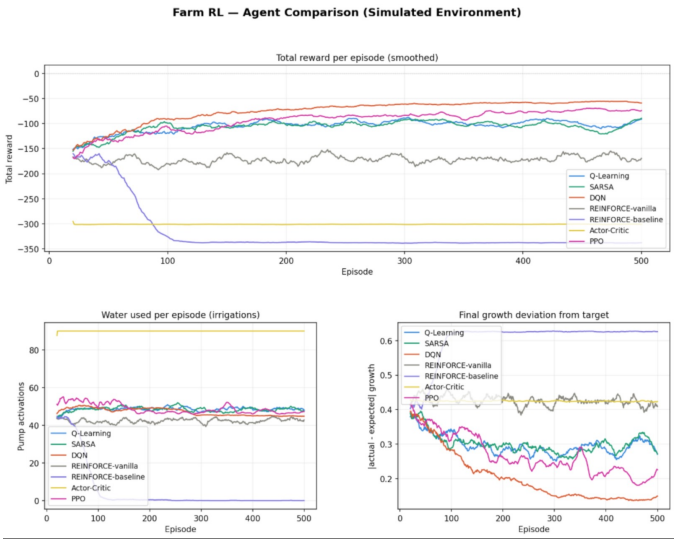

Fig. 2. Training Reward Curves for all seven evaluated algorithms over 500 episodes showing the convergence of DQN and PPO versus the plateauing of tabular methods.

C. Limitations

The action space is binary (on/off). A continuous action space would improve precision. Plant health is currently estimated via math rather than direct visual observation and weather data is tied to North Bangalore rather than the specific farm location.

VII. CONCLUSION AND FUTURE SCOPE

ARIA demonstrated that Deep RL (specifically DQN) can effectively manage complex irrigation cycles across seasons. Three major lessons emerged: Deep RL is necessary for continuous multidimensional spaces. Reward shaping is the primary challenge in RL design. Bridiging the gap between simulation and deployment is non negotiable.

Future work includes connecting the YOLO pipeline directly to the agent's state vector, moving to an A3C architecture for multizone farming and conducting a multi-season field trial with live weather data.

Disclaimer: We have made this report using AI. It was used for formatting, helping in the Latex code and making the text more presentable. We would say approximately 30% of this report credit goes to Gemini :)